

SETUP AND ACCESS

Set up Claude Code for your organization

A decision map for administrators deploying Claude Code, covering API providers, managed settings, policy enforcement, usage monitoring, and data handling.

Claude Code enforces organization policy through managed settings that take precedence over local developer configuration. You deliver those settings from the Claude admin console, your mobile device management (MDM) system, or a file on disk. The settings control which tools, commands, servers, and network destinations Claude can reach.

This page walks through the deployment decisions in order. Each row links to the section below and to the reference page for that area.

 SSO, SCIM provisioning, and seat assignment are configured at the Claude account level. See the [Claude Enterprise Administrator Guide](#) and [seat assignment](#) for those steps.

Decision	What you're choosing	Reference
Choose your API provider	Where Claude Code authenticates and how it's billed	Authentication , Bedrock , Vertex AI , Foundry
Decide how settings reach devices	How managed policy reaches developer machines	Server-managed settings , Settings files
Decide what to enforce	Which tools, commands, and integrations are allowed	Permissions , Sandboxing
Set up usage visibility	How you track spend and adoption	Analytics , Monitoring , Costs
Review data handling	Data retention and compliance posture	Data usage , Security

Choose your API provider

Claude Code connects to Claude through one of several API providers. Your choice affects billing, authentication, which compliance posture you inherit, and which Claude Code features your developers can use.

Provider	Choose this when
Claude for Teams / Enterprise	You want Claude Code and claude.ai under one per-seat subscription with no infrastructure to run. This is the default recommendation.
Claude Console	You're API-first or want pay-as-you-go billing
Amazon Bedrock	You want to inherit existing AWS compliance controls and billing
Google Vertex AI	You want to inherit existing GCP compliance controls and billing
Microsoft Foundry	You want to inherit existing Azure compliance controls and billing

Some Claude Code features require a claude.ai account. [Claude Code on the web](#), [Routines](#), [Code Review](#), [Remote Control](#), and the [Chrome extension](#) aren't available through Console API keys or cloud-provider credentials alone. If you deploy through Bedrock, Vertex, or Foundry, plan whether developers also need Claude for Teams or Enterprise seats. Each feature page lists its plan requirements.

For the full provider comparison covering authentication, regions, and feature parity, see the [enterprise deployment overview](#). Each provider's auth setup is in [Authentication](#).

Proxy and firewall requirements in [Network configuration](#) apply regardless of provider. If you want a single endpoint in front of multiple providers or centralized request logging, see [LLM gateway](#).

Decide how settings reach devices

Managed settings define policy that takes precedence over local developer configuration. Claude Code checks the four sources below in priority order and applies the first one that returns a non-empty configuration.

Mechanism	Delivery	Priority	Platforms
Server-managed	claude.ai admin console	Highest	All
plist / registry policy	macOS: <code>com.anthropic.claudecode</code> plist Windows: <code>HKLM\SOFTWARE\Policies\ClaudeCode</code>	High	macOS, Windows
File-based managed	macOS: <code>/Library/Application Support/ClaudeCode/managed-settings.json</code> Linux and WSL: <code>/etc/claude-code/managed-settings.json</code> Windows: <code>C:\Program Files\ClaudeCode\managed-</code>	Medium	All

Mechanism	Delivery	Priority	Platforms
	settings.json		
Windows user registry	HKCU\SOFTWARE\Policies\ClaudeCode	Lowest	Windows only

Server-managed settings reach devices at authentication time and refresh hourly during active sessions, with no endpoint infrastructure. They require a Claude for Teams or Enterprise plan, so deployments on other providers need one of the file-based or OS-level mechanisms instead.

If your organization mixes providers, configure server-managed settings for claude.ai users plus a file-based or plist/registry fallback so other users still receive managed policy.

The plist and HKLM registry locations work with any provider and resist tampering because they require admin privileges to write. The Windows user registry at HKCU is writable without elevation, so treat it as a convenience default rather than an enforcement channel.

By default, WSL reads only the Linux file path at `/etc/claude-code` . To extend your Windows registry and `C:\Program Files\ClaudeCode` policy to WSL on the same machine, set wslinheritsWindowsSettings: true in either of those admin-only Windows sources.

Whichever mechanism you choose, managed values take precedence over user and project settings. Array settings such as `permissions.allow` and `permissions.deny` merge entries from all sources, so developers can extend managed lists but not remove from them. For two exceptions, `fallbackModel` and `availableModels` , the managed value replaces lower layers rather than merging.

See Server-managed settings and Settings files and precedence.

Decide what to enforce

Managed settings can lock down tools, sandbox execution, restrict MCP servers and plugin sources, and control which hooks run. Each row is a control surface with the setting keys that drive it.

Control	What it does	Key settings
<u>Permission rules</u>	Allow, ask, or deny specific tools and commands	<code>permissions.allow</code> , <code>permissions.deny</code>
<u>Permission lockdown</u>	Only managed permission rules apply; disable <code>--dangerously-skip-permissions</code>	<code>allowManagedPermissionRulesOnly</code> , <code>permissions.disableBypassPermis</code>

Control	What it does	Key settings
		<code>sionsMode</code>
<u>Sandboxing</u>	OS-level filesystem and network isolation with domain allowlists	<code>sandbox.enabled</code> , <code>sandbox.network.allowedDomains</code>
<u>Managed policy</u> <u>CLAUDE.md</u>	Org-wide instructions loaded in every session, can't be excluded	File at the managed policy path
<u>MCP server control</u>	Restrict which MCP servers users can add or connect to, or deploy a fixed set	<code>allowedMcpServers</code> , <code>deniedMcpServers</code> , <code>allowManagedMcpServersOnly</code> , or a deployed <code>managed-mcp.json</code> file
<u>Plugin marketplace control</u>	Restrict which marketplace sources users can add and install from	<code>strictKnownMarketplaces</code> , <code>blockedMarketplaces</code>
<u>Customization lockdown</u>	Block skills, agents, hooks, and MCP servers from user and project sources, so they can only come from plugins or managed settings	<code>strictPluginOnlyCustomization</code>
<u>Hook restrictions</u>	Only managed hooks load; restrict HTTP hook URLs	<code>allowManagedHooksOnly</code> , <code>allowedHttpRequests</code>
<u>Disable agent view</u>	Turn off <code>claude agents</code> , <code>--bg</code> , <code>/background</code> , and the on-demand supervisor	<code>disableAgentView</code>
<u>Model restrictions</u>	<code>availableModels</code> filters which models appear in the picker. Adding <code>enforceAvailableModels</code> also constrains the auto-selected default model. See <u>surface coverage</u> for how this setting reaches the CLI, web, and IDE	<code>availableModels</code> , <code>enforceAvailableModels</code>
<u>Version floor</u>	Prevent auto-update from installing below an org-wide minimum	<code>minimumVersion</code>
<u>Required version range</u>	Refuse to start at all when the running version is outside an org-approved range. Stronger than <code>minimumVersion</code> , which only blocks downgrades	<code>requiredMinimumVersion</code> , <code>requiredMaximumVersion</code>

Permission rules and sandboxing cover different layers. Denying WebFetch blocks Claude’s fetch tool, but if Bash is allowed, `curl` and `wget` can still reach any URL. Sandboxing closes that gap with a network domain allowlist enforced at the OS level.

For the threat model these controls defend against, see Security.

Set up usage visibility

Choose monitoring based on what you need to report on.

Capability	What you get	Availability	Where to start
Usage monitoring	OpenTelemetry export of sessions, tools, and tokens	All providers	<u>Monitoring usage</u>
Analytics dashboard	Per-user metrics, contribution tracking, leaderboard	Anthropic only	<u>Analytics</u>
Cost tracking	Spend limits, rate limits, and usage attribution	Anthropic only	<u>Costs</u>

Cloud providers expose spend through AWS Cost Explorer, GCP Billing, or Azure Cost Management. Claude for Teams and Enterprise plans include a usage dashboard at [claude.ai/analytics/claude-code](#).

Review data handling

On Team, Enterprise, Claude API, and cloud provider plans, Anthropic doesn’t train models on your code or prompts. Your API provider determines retention and compliance posture.

Topic	What to know	Where to start
Data usage policy	What Anthropic collects, how long it’s retained, what’s never used for training	<u>Data usage</u>
Zero Data Retention (ZDR)	Nothing stored after the request completes. Available to qualified accounts on Claude for Enterprise	<u>Zero data retention</u>
Security architecture	Network model, encryption, authentication, audit trail	<u>Security</u>

If you need request-level audit logging or to route traffic by data sensitivity, place an [LLM gateway](#) between developers and your provider. For regulatory requirements and certifications, see [Legal and compliance](#).

Verify and onboard

After configuring managed settings, have a developer run `/status` inside Claude Code. On the **Status** tab, the `Setting sources` line shows `Enterprise managed settings` followed by the source in parentheses, one of `(remote)` , `(plist)` , `(HKLM)` , `(HKCU)` , or `(file)` . See [Verify active settings](#).

Share these resources to help developers get started:

- **Quickstart**: first-session walkthrough from install to working with a project
- **Common workflows**: patterns for everyday tasks like code review, refactoring, and debugging
- **Claude 101** and **Claude Code in Action**: self-paced Anthropic Academy courses

For login issues, point developers to **authentication troubleshooting**. The most common fixes are:

- Run `/logout` then `/login` to switch accounts
- Run `claude update` if the enterprise auth option is missing
- Restart the terminal after updating

If a developer sees “You haven’t been added to your organization yet,” their seat doesn’t include Claude Code access and needs to be updated in the admin console.

Next steps

With provider and delivery mechanism chosen, move on to detailed configuration:

- **Server-managed settings**: deliver managed policy from the Claude admin console
- **Settings reference**: every setting key, file location, and precedence rule
- **Monorepos and large repos**: per-directory configuration patterns for organizations deploying into a monorepo
- **Amazon Bedrock**, **Google Vertex AI**, **Microsoft Foundry**: provider-specific deployment
- **Claude Enterprise Administrator Guide**: SSO, SCIM, seat management, and rollout playbook